

BDA400
Data Science Tools and Techniques

Assignment 5

**Technical Analysis using
R – Development Phase**

Student: Kamil Adebumola

Course: BDA400

GitHub Repository: [<https://github.com/camtop007/TechnicalAnalysis>]

Report Contents

1. Introduction
2. Simple Moving Average (SMA)
3. Exponential Moving Average (EMA)
4. Moving Average Convergence Divergence (MACD)
5. Standard Deviation
6. Linear Regression
7. Relative Strength Index (RSI)
8. Stochastic RSI (StochRSI)
9. Crossover
10. Crossunder
11. Testing and Validation
12. How to Run the Code
13. Conclusion
14. GitHub Repository Link

1. Introduction

This report documents the development phase of the BDA400 Technical Analysis using R project. The objective is to implement core technical-analysis indicators and trading-signal functions directly in R using standard language features rather than relying on pre-built technical-analysis libraries. The work covers Simple Moving Average, Exponential Moving Average, MACD, population standard deviation, simple linear regression, Relative Strength Index, Stochastic RSI, crossover, and crossunder logic.

Each indicator is implemented as a separate R function using the naming convention specified in the assignment. The functions are designed to be transparent, testable, and reusable. Validation is performed with small sample datasets so that the expected behaviour can be checked manually and the results can be inspected before applying the functions to historical stock-market data.

Item	Details
Implementation approach	Base/core R logic is used for the calculations. Existing technical-analysis indicator functions are not used.
File organization	Each indicator is stored in a separate .R script, supported by a master test script.
Validation approach	Known sample data, manual spot-checks, input validation, and cross-function consistency checks.
Primary outcome	A reusable set of technical-analysis functions that can be integrated with stock data from the earlier project stage.

2. Simple Moving Average (SMA)

The Simple Moving Average smooths a numeric series by calculating the arithmetic mean of a fixed-size moving window. Every observation within the selected period receives equal weight. The implementation moves across the input vector one window at a time and stores each calculated average.

Item	Details
Function	sma(data, period)
Inputs	Numeric vector and positive integer period.
Output	Numeric vector of moving-average values.
Core logic	For each window: sum the values and divide by the period.

```
sma <- function(data, period) {  
  if (!is.numeric(data)) stop("Data must be numeric")  
  if (period <= 0 || period != as.integer(period)) stop("Period must be a positive integer")  
  if (length(data) < period) stop("Data length should be greater than or equal to the period")  
}
```

```

sma_values <- numeric(length(data) - period + 1)
for (i in 1:(length(data) - period + 1)) {
  current_window <- data[i:(i + period - 1)]
  sma_values[i] <- sum(current_window) / period
}
return(sma_values)
}

```

Test example: data = c(10, 12, 15, 20, 18, 22, 25, 24, 21), period = 3. The first result is (10 + 12 + 15) / 3 = 12.3333. Subsequent results are produced by shifting the window one position at a time.

3. Exponential Moving Average (EMA)

The Exponential Moving Average gives greater weight to more recent observations, making it more responsive to recent changes than the SMA. The smoothing multiplier is calculated as $2 / (\text{period} + 1)$. The first EMA is initialized with the first data value, and each later value is calculated recursively from the current observation and previous EMA.

Item	Details
Function	ema(data, period)
Multiplier	$2 / (\text{period} + 1)$
Output	EMA value for every input observation.
Dependency	None; implemented directly in base R.

```

ema <- function(data, period) {
  if (!is.numeric(data)) stop("Data must be numeric")
  if (length(data) == 0) stop("Data cannot be empty")
  if (period <= 0 || period != as.integer(period)) stop("Period must be a positive integer")

  multiplier <- 2 / (period + 1)
  ema_values <- numeric(length(data))
  ema_values[1] <- data[1]

  if (length(data) > 1) {
    for (i in 2:length(data)) {
      ema_values[i] <- (data[i] - ema_values[i - 1]) * multiplier + ema_values[i - 1]
    }
  }
  return(ema_values)
}

```

For a period of 3, the multiplier is 0.5. Using the sample series, the first EMA is 10, the second is 11, and the third is 13. These values confirm that recent observations receive more influence than older ones.

4. Moving Average Convergence Divergence (MACD)

MACD measures trend and momentum by comparing a short-period EMA with a long-period EMA. The MACD line is the short EMA minus the long EMA. A signal line is then calculated as an EMA of the MACD line, and the histogram is the difference between the MACD line and the signal line.

Item	Details
Function	macd(data, short_period, long_period, signal_period)
Returns	List containing macd_line, signal_line, and histogram.
Dependency	Uses the custom ema() function.
Interpretation	Positive MACD generally indicates bullish momentum; negative MACD generally indicates bearish momentum.

```
macd <- function(data, short_period, long_period, signal_period) {  
  if (!is.numeric(data)) stop("Data must be numeric")  
  if (short_period <= 0 || long_period <= 0 || signal_period <= 0) stop("All periods must be  
positive integers")  
  if (short_period >= long_period) stop("short_period must be less than long_period")  
  
  short_ema <- ema(data, short_period)  
  long_ema <- ema(data, long_period)  
  macd_line <- short_ema - long_ema  
  signal_line <- ema(macd_line, signal_period)  
  histogram <- macd_line - signal_line  
  
  return(list(macd_line = macd_line, signal_line = signal_line, histogram = histogram))  
}
```

The sample test with data c(100, 105, 110, 115, 120, 125, 130), short period 3, long period 5, and signal period 2 produced a rising positive MACD line. The histogram was verified by subtracting the signal line from the MACD line at individual observations.

5. Standard Deviation

The standard-deviation function measures the dispersion of a numeric series around its mean. The assignment specifies the population formulation, so the squared deviations are divided by n rather than $n - 1$. The calculation is performed manually instead of calling R's built-in `sd()` function.

Item	Details
Function	<code>stdev(data)</code>
Formula	Square root of the mean squared deviation from the arithmetic mean.
Output	Single numeric population standard deviation.
Reason for manual implementation	Matches the assignment formula and demonstrates the underlying algorithm.

```
stdev <- function(data) {  
  if (!is.numeric(data)) stop("Data must be numeric")  
  if (length(data) == 0) stop("Data cannot be empty")  
  
  mean_value <- sum(data) / length(data)  
  diff_values <- data - mean_value  
  squared_diff <- diff_values^2  
  variance <- sum(squared_diff) / length(squared_diff)  
  standard_deviation <- sqrt(variance)  
  return(standard_deviation)  
}
```

A useful sanity check is a constant vector such as `c(5, 5, 5, 5, 5)`, which should return a standard deviation of 0 because there is no dispersion.

6. Linear Regression

The `linreg()` function fits a simple straight-line model to a selected portion of a numeric sequence. The function determines the subset using `regressionLength` and `regressionOffset`, creates sequential index values, and calculates the slope and intercept from the covariance-style numerator and squared-deviation denominator. Predicted values are then generated from the fitted line.

Item	Details
Function	<code>linreg(regressionSource, regressionLength, regressionOffset)</code>
Returns	List containing slope, intercept, and <code>predicted_values</code> .
Validation	Regression length cannot exceed the data length; offset must be smaller than the regression length.

Model	Predicted value = intercept + slope x index.
--------------	--

```
linreg <- function(regressionSource, regressionLength, regressionOffset) {
  n <- length(regressionSource)
  if (!is.numeric(regressionSource)) stop("regressionSource must be numeric")
  if (regressionLength > n) stop("regressionLength cannot be greater than the number of
elements in regressionSource")
  if (regressionOffset >= regressionLength) stop("regressionOffset must be less than
regressionLength")

  start_index <- max(1, n - regressionLength + regressionOffset)
  end_index <- min(n, n - regressionOffset)
  source_subset <- regressionSource[start_index:end_index]
  index_values <- 1:length(source_subset)

  mean_index <- sum(index_values) / length(index_values)
  mean_source <- sum(source_subset) / length(source_subset)
  numerator <- sum((index_values - mean_index) * (source_subset - mean_source))
  denominator <- sum((index_values - mean_index)^2)
  if (denominator == 0) stop("Cannot calculate regression: denominator is zero")

  slope <- numerator / denominator
  intercept <- mean_source - slope * mean_index
  predicted_values <- slope * index_values + intercept
  return(list(slope = slope, intercept = intercept, predicted_values = predicted_values))
}
```

Testing with the last five values of `c(10, 12, 14, 16, 18, 20, 22, 24, 26, 28)` produces a slope of 2. The fitted values reproduce the sequence 20, 22, 24, 26, 28, providing a clear validation of the regression implementation.

7. Relative Strength Index (RSI)

RSI is a momentum oscillator that compares average gains with average losses and expresses the result on a 0-100 scale. The implementation calculates consecutive changes, separates gains and losses, computes initial averages, and then applies Wilder-style smoothing. Values above 70 are traditionally interpreted as overbought and values below 30 as oversold.

Item	Details
Function	<code>rsi(data, period)</code>
Core steps	Differences -> gains/losses -> initial averages -> Wilder smoothing -> RS -> RSI.

Output	Numeric vector aligned with the input length, with initial NA values.
Range	0 to 100 when defined.

```

rsi <- function(data, period) {
  if (!is.numeric(data)) stop("Data must be numeric")
  if (length(data) <= period) stop("Data length must be greater than period")

  diff_values <- diff(data)
  gains <- numeric(length(diff_values))
  losses <- numeric(length(diff_values))

  for (i in 1:length(diff_values)) {
    if (diff_values[i] > 0) gains[i] <- diff_values[i]
    else losses[i] <- abs(diff_values[i])
  }

  avg_gain <- sum(gains[1:period]) / period
  avg_loss <- sum(losses[1:period]) / period
  rsi_values <- rep(NA_real_, length(data))

  for (i in (period + 1):length(data)) {
    avg_gain <- (avg_gain * (period - 1) + gains[i - 1]) / period
    avg_loss <- (avg_loss * (period - 1) + losses[i - 1]) / period
    if (avg_loss == 0) rsi_values[i] <- 100
    else {
      rs <- avg_gain / avg_loss
      rsi_values[i] <- 100 - (100 / (1 + rs))
    }
  }
  return(rsi_values)
}

```

The assignment sample data `c(45, 50, 48, 55, 52, 49, 58, 60, 65, 62)` with period 5 is used to validate the function. Initial NA values are expected because the indicator requires prior observations before a smoothed RSI can be produced.

8. Stochastic RSI (StochRSI)

Stochastic RSI applies stochastic normalization to RSI values so that momentum can be viewed relative to the recent RSI range. The normalized value is calculated from the current RSI, the lowest RSI, and the highest RSI in the selected window. The function then derives %K and %D smoothing lines using the custom `sma()` function.

Item	Details
Function	stoch_rsi(data, period, k_period, d_period)
Dependencies	Uses custom rsi() and sma() functions.
Range	Normalized StochRSI values are typically between 0 and 1.
Returns	List containing stoch_rsi, k, and d.

```

stoch_rsi <- function(data, period, k_period, d_period) {
  rsi_values <- rsi(data, period)
  stoch_values <- rep(NA_real_, length(rsi_values))

  for (i in period:length(rsi_values)) {
    start_index <- i - period + 1
    rsi_window <- rsi_values[start_index:i]
    valid_values <- rsi_window[!is.na(rsi_window)]

    if (length(valid_values) > 0 && !is.na(rsi_values[i])) {
      rsi_lowest <- min(valid_values)
      rsi_highest <- max(valid_values)
      if (rsi_highest != rsi_lowest) {
        stoch_values[i] <- (rsi_values[i] - rsi_lowest) / (rsi_highest - rsi_lowest)
      }
    }
  }

  valid_stoch <- stoch_values[!is.na(stoch_values)]
  k_values <- sma(valid_stoch, k_period)
  d_values <- sma(k_values, d_period)
  return(list(stoch_rsi = stoch_values, k = k_values, d = d_values))
}

```

A longer synthetic price sequence is used for testing because RSI and its subsequent smoothing steps require sufficient observations. Initial missing values are expected and are not treated as calculation errors.

9. Crossover

The crossover function compares two equal-length series and identifies relationship changes. It returns "Up" when the first series crosses from below or equal to the second series to above it. The assignment pseudocode also permits "Down" when the first series moves from above or equal to below, with "None" used otherwise.

Item	Details
------	---------

Function	crossover(arr1, arr2)
Validation	Both arrays must have identical length.
Signals	Up, Down, or None.
Use case	Can be applied to moving averages or other indicator lines to detect potential trend changes.

```

crossover <- function(arr1, arr2) {
  if (length(arr1) != length(arr2)) stop("Both arrays should have the same length")
  crossover_signals <- rep("None", length(arr1))

  for (i in 2:length(arr1)) {
    if (arr1[i] > arr2[i] && arr1[i - 1] <= arr2[i - 1]) crossover_signals[i] <- "Up"
    else if (arr1[i] < arr2[i] && arr1[i - 1] >= arr2[i - 1]) crossover_signals[i] <- "Down"
    else crossover_signals[i] <- "None"
  }
  return(crossover_signals)
}

```

The supplied sample arrays are used to confirm that the function recognizes where the relationship between the two series changes. A separate unequal-length test confirms that invalid inputs are rejected.

10. Crossunder

The crossunder function identifies the specific event where the first series was previously greater than or equal to the second series and then becomes less than it. The function returns "True" at the crossunder point, "False" at other subsequent observations, and "None" for the first observation because a prior comparison does not exist.

Item	Details
Function	crossunder(arr1, arr2)
Validation	Both arrays must have the same length.
Signal condition	arr1[i] < arr2[i] and arr1[i-1] >= arr2[i-1].
Output	Character vector containing None, True, and False.

```

crossunder <- function(arr1, arr2) {
  if (length(arr1) != length(arr2)) stop("Both arrays should have the same length")
  crossunder_signals <- rep("False", length(arr1))
  crossunder_signals[1] <- "None"

  for (i in 2:length(arr1)) {

```

```

    if (arr1[i] < arr2[i] && arr1[i - 1] >= arr2[i - 1]) crossunder_signals[i] <- "True"
    else crossunder_signals[i] <- "False"
  }
  return(crossunder_signals)
}

```

The assignment sample arrays contain no downward crossing after the initial upward crossover, so an additional test series is useful to confirm a true crossunder event. For example, `arr1 = c(10, 12, 14, 16, 13, 9, 7)` and `arr2 = c(8, 9, 10, 11, 12, 10, 8)` produce a crossunder when 9 falls below 10 after the previous value 13 was above 12.

11. Testing and Validation

Testing is performed at two levels: individual function tests and an integration test that sources every script in dependency order. The master test script uses known sample data so that results can be checked manually or logically. The following checks are used to confirm correctness:

- SMA: manually verify the first moving average from the first window.
- EMA: verify the multiplier and the first several recursive calculations.
- MACD: confirm that `histogram = MACD line - signal line` for selected observations.
- Standard deviation: confirm that a constant vector returns 0.
- Linear regression: use a perfectly linear sequence and verify slope, intercept, and fitted values.
- RSI: confirm that defined values remain between 0 and 100 and that early values are NA.
- StochRSI: confirm normalized values remain between 0 and 1 when defined.
- Crossover/Crossunder: test both valid transitions and unequal-length error handling.

Master integration test - test_all_indicators.R

```

source("sma.R")
source("ema.R")
source("macd.R")
source("stdev.R")
source("linreg.R")
source("rsi.R")
source("stoch_rsi.R")
source("crossover.R")
source("crossunder.R")

```

Run representative tests after the functions are loaded.

A successful integration test should execute without object-not-found or file-not-found errors. If `source()` reports that a file cannot be opened, the working directory should be checked with `getwd()` and `list.files()` before rerunning the test.

12. How to Run the Code

All scripts should be placed in the same Assignment 5 folder. The recommended execution process is:

- Open the Assignment 5 folder as the active RStudio working directory or RStudio Project.
- Confirm the location with `getwd()`.
- Confirm the scripts are visible with `list.files()`.
- Load individual scripts with `source("filename.R")` or run the master test script.
- Run the sample tests and inspect the Console output for errors or unexpected values.

```
getwd()
list.files()

# Load all functions and tests
source("test_all_indicators.R")
```

Recommended folder structure:

```
TechnicalAnalysis/
├── Assignment5/
│   ├── sma.R
│   ├── ema.R
│   ├── macd.R
│   ├── stdev.R
│   ├── linreg.R
│   ├── rsi.R
│   ├── stoch_rsi.R
│   ├── crossover.R
│   ├── crossunder.R
│   ├── test_all_indicators.R
│   └── Assignment5_Report.docx
```

13. Conclusion

This assignment developed a complete set of core technical-analysis functions in R without relying on pre-built indicator libraries. The implementation progressed from basic smoothing methods to momentum indicators, regression, dispersion measurement, and trading-signal logic. Building the functions manually reinforced the mathematical reasoning behind each indicator and made the dependencies between functions explicit.

The testing process also highlighted the importance of input validation, file organization, dependency order, and reproducible execution. The resulting scripts form a modular foundation that can be connected to historical stock-market data in the broader Technical Analysis project. Before final submission, all scripts should be rerun from a clean R session to confirm that the complete project executes successfully and that the repository contains the required files.

14. GitHub Repository Link

Repository URL: <https://github.com/camtop007/TechnicalAnalysis>

The repository should contain the separate indicator scripts, the master test script, and the final report. The repository link should be checked in a browser before submission to ensure that it is accessible to the evaluator.